Task 1: Data Loading and Initial Inspection

Task Description

Load the data from the V002.json file into a Pandas DataFrame. Create a separate Series for the video URL. Then, display the first 5 rows, the data types of all columns, and the total number of sentences.

Key Pandas Concepts

pd.read_json(), DataFrame/Series creation, .head(), .info(), .shape

```
import pandas as pd
import numpy as np
import json
from collections import Counter

v002_path = "V002.json"

# Load the JSON data
with open(v002_path, 'r', encoding='utf-8') as file:
    raw_data = json.load(file)

# The data is nested under "V002.mp4" key
video_key = "V002.mp4"
video_data = raw_data[video_key]

# Create main DataFrame from instances
df = pd.DataFrame(video_data['instances'])

#todo:

#1. Display first 5 rows
display(df.head())

#2. Display total number of sentences
print(f"Total number of sentences: {df.shape[0]}")

#3. Display columns
print(f"Columns: {list(df.columns)}")
```

```
                                            sentence  start   end
split
0                                  □□□□□□□□ □□□□□□□  260   281
train
1  □□□□□□ □ □□□□□□ □□□□□□□□□□□ □□□ □ □□□□□□ □□□□□...    282   518
train
```

```
2  □□ □□□□□□ □□□□□□□ □□□□□□ □□□□□□□□□□□□ □□□□□□□□...    592   742
train
3  □□□□ □□□□□ □□□□□□□□□ □□□□□□□□□ □□□□□□ □□□□□□□□...   1361  1643
train
4  □□ □□□□ □□□□□□ □□□□□□ □□□□□□ □□□□□□□□ □□ □□□□□...   1643  1935
test

Total number of sentences: 42
Columns: ['sentence', 'start', 'end', 'split']
```

Task 2: Feature Extraction and Manipulation (Time Features)

Task Description

Convert frame numbers to seconds using video FPS metadata. Create new columns for start time, end time, and duration in seconds.

Key Pandas Concepts

Arithmetic operations on columns, creating new columns, vectorized operations

```python
# Task 2 – Convert frame numbers to seconds

# 1. Extract fps from video metadata
fps = video_data["fps"]


# 2. Convert start and end frames to seconds
df["start_sec"] = df["start"] / fps
df["end_sec"]   = df["end"] / fps

#todo:
# 3. Calculate duration in seconds (Subtract the end_sec from the
start_sec)
df["duration_sec"] = df["end_sec"] - df["start_sec"]

# 4. Display results
df[["start", "end", "start_sec", "end_sec", "duration_sec"]].head()
```

```
   start   end  start_sec    end_sec  duration_sec
0    260   281   8.666667   9.366667      0.700000
1    282   518   9.400000  17.266667      7.866667
2    592   742  19.733333  24.733333      5.000000
3   1361  1643  45.366667  54.766667      9.400000
4   1643  1935  54.766667  64.500000      9.733333
```

Task 3: Data Filtering (Conditional Selection)

Task Description

Create two new DataFrames:

- df_long: Contains only sentences that are longer than 5 seconds
- df_train: Contains only sentences where the split column is 'train' Report the number of sentences in df_long and the average duration of sentences in df_train.

Key Pandas Concepts

Boolean masking/conditional filtering, .loc[], .mean(), .count()

```
# Task 3 – Data Filtering

# 1. Sentences longer than 5 seconds
df_long = df.loc[df["duration_sec"] > 5] #todo

# 2. Sentences belonging to the 'train' split
df_train = df.loc[df["split"] == "train"]

# 3. Reporting
num_long_sentences = df_long.shape[0]
avg_duration_train = df_train["duration_sec"].mean()

print("Number of sentences longer than 5 seconds:",
num_long_sentences)
print("Average duration of 'train' sentences:", avg_duration_train)
```

```
Number of sentences longer than 5 seconds: 31
Average duration of 'train' sentences: 7.031034482758617
```

Task 4: Text Data Preprocessing (String Operations)

Task Description

Clean the sentence column by performing the following steps:

- Remove leading/trailing whitespace
- Check for and count how many sentences contain specific words
- Create a new column sentence_length containing the character count of each sentence

Key Pandas Concepts

.str.strip(), .str.contains(), .sum(), .str.len()

```
# Task 4 — Text Data Preprocessing

# 1. Remove leading/trailing whitespace
df['sentence'] = df['sentence'].str.strip()

# 2. Count sentences containing the word "তিনি "
count = df['sentence'].str.contains("□□□□").sum()
print("Number of sentences containing 'তিনি ':", count)

# 3. Create a new column for sentence length (character count)
df['sentence_length'] = df['sentence'].str.len() #todo

# 4. Preview the updated dataframe
df.head()
```

```
Number of sentences containing 'তিনি ': 3

                                            sentence  start   end  
split  \
0                                 আসসালামু আলাইকুম    260   281   
train
1  বিটিভি ও বিটিভি ওয়ার্ল্ডের দেশ ও জনপদের খবরের           ...    282   518   
train
2  আজ পাবনার ঈশ্বরদী উপজেলা মুক্তিযোদ্ধা কমান্ডের           ...    592   742   
train
3  তিনি গতকাল মাদারীপুর স্বাধীনতা অঙ্গনে মাদারীপু           ...   1361  1643   
train
4  এই সময় বইমেলা উদযাপন কমিটির আহ্বায়ক সহ সরকার     ...   1643  1935   
test

   start_sec    end_sec  duration_sec  sentence_length  
0   8.666667   9.366667      0.700000               16  
1   9.400000  17.266667      7.866667              133  
2  19.733333  24.733333      5.000000              101  
3  45.366667  54.766667      9.400000              137  
4  54.766667  64.500000      9.733333              117  
```

Task 5: Aggregation and Grouping (Split Analysis)

Task Description

Group the DataFrame by the split column ('train', 'test', 'val'). For each group, calculate and display:

- The total duration of all sentences
- The number of sentences
- The average duration per sentence

Key Pandas Concepts

.groupby(), .agg(), named aggregation

```
# Task 5 — Grouping and Aggregation by 'split'

# Group by the 'split' column and compute aggregations
split_stats = df.groupby('split').agg(
    total_duration=('duration_sec', 'sum'),
    sentence_count=('sentence', 'count'),
    avg_duration=('duration_sec', 'mean')
) #todo

# Display the aggregated results
split_stats
```

```
       total_duration  sentence_count  avg_duration
split
test        53.666667               8      6.708333
train      203.900000              29      7.031034
val         38.333333               5      7.666667
```

Task 6: Missing Value Analysis and Handling

Task Description

In real-world datasets, it is common to encounter missing values. In this task, you will:

- Simulate a missing value in the sentence column by setting the first row to None
- Check how many missing values exist in the sentence column
- Replace missing values with the placeholder string: "[MISSING SENTENCE]"

Key Pandas Concepts

.loc[] for value assignment, .isnull(), .sum(), .fillna()

```
# Task 6 — Missing Value Analysis and Handling

# 1. Simulate a missing value in the first row
# df.loc[0, "sentence"] = None

# 2. Count missing values in the 'sentence' column
missing_count = df['sentence'].isnull().sum() #todo
print("Number of missing sentences:", missing_count)

# 3. Replace missing values with '[MISSING SENTENCE]'
df['sentence'] = df['sentence'].fillna("[MISSING SENTENCE]")

# 4. Preview the updated DataFrame
df.head()
```

```
Number of missing sentences: 0
```

	sentence	start	end	split
0	□□□□□□□□ □□□□□□□	260	281	train
1	□□□□□□ □ □□□□□□ □□□□□□□□□□□ □□□ □ □□□□□□ □□□□□...	282	518	train
2	□□ □□□□□□ □□□□□□□ □□□□□□ □□□□□□□□□□□ □□□□□□□□...	592	742	train
3	□□□□ □□□□□ □□□□□□□□□ □□□□□□□□□ □□□□□□ □□□□□□□□...	1361	1643	train
4	□□ □□□□ □□□□□□ □□□□□□ □□□□□□ □□□□□□□□ □□ □□□□□...	1643	1935	test

	start_sec	end_sec	duration_sec	sentence_length
0	8.666667	9.366667	0.700000	16
1	9.400000	17.266667	7.866667	133
2	19.733333	24.733333	5.000000	101
3	45.366667	54.766667	9.400000	137
4	54.766667	64.500000	9.733333	117

Task 7: Indexing and Hierarchical Data Access

Task Description

In this task, you will practice indexing and hierarchical data access in Pandas.

- Set the split column as the DataFrame's index

- Use the .loc[] accessor to select specific rows/columns:
  - All rows belonging to the 'test' split
  - Only start_sec and end_sec columns for rows in the 'val' split

Key Pandas Concepts

.set_index(), Hierarchical indexing, .loc[] for row and column selection

```python
# Task 7 — Indexing and Hierarchical Data Access

# 1. Set 'split' column as the index
df_indexed = df.set_index("split")

# 2. Select all rows belonging to the 'test' split
df_test = df_indexed.loc["test"]
print("All rows in 'test' split:")
display(df_test)

# 3. Select only start_sec and end_sec for the 'val' split
df_val_times = df_indexed.loc["val", ["start_sec", "end_sec"]] #todo
print("\nStart and End times for 'val' split:")
display(df_val_times)
```

```
All rows in 'test' split:

                                                            sentence  start    end  \
split

test   এই সময় বইমেলা উদযাপন কমিটির আহ্বায়ক সহ সরকার       ...   1643   1935

test   ইনভেস্টমেন্ট কর্পোরেশন অব বাংলাদেশ আইসিবি বিনি       ...   6995
7228
test       সভায় বিপুল সংখ্যক শেয়ারহোল্ডার অংশগ্রহণ করেন           7777   7886

test   এইসব কর্মসূচিতে স্থানীয় জনপ্রতিনিধি ও সরকারি           ...  12404
12597
test   দেশের বিভিন্ন জেলা থেকে একশো ছিয়ানব্বই টি ইউন         ...  14993  15212

test   দিনটি উপলক্ষে নেত্রকোণার বিভিন্ন সামাজিক সাংস্          ...   15708
15928
test                বাজার নিজের দোকান থেকে তাকে আটক করা হয়    19183  19314

test   উদ্বোধনী খেলায় কালীগঞ্জ উপজেলা চার এক গোলে দে        ...  20944  21157


        start_sec     end_sec  duration_sec  sentence_length
split
test    54.766667   64.500000      9.733333              117
test   233.166667  240.933333      7.766667              121
```

```
test   259.233333  262.866667      3.633333                46
test   413.466667  419.900000      6.433333                86
test   499.766667  507.066667      7.300000                97
test   523.600000  530.933333      7.333333                81
test   639.433333  643.800000      4.366667                39
test   698.133333  705.233333      7.100000                71


Start and End times for 'val' split:

         start_sec     end_sec
split
val    240.933333  247.466667
val    247.466667  259.233333
val    392.500000  396.066667
val    476.233333  487.000000
val    656.800000  662.500000
```

Task 8: Handling Categorical Data

Task Description

In this task, you will optimize the DataFrame for memory efficiency by converting the split column into a categorical data type.

Steps:

- Check the memory usage of the DataFrame before conversion
- Convert the split column to Categorical using .astype('category')
- Check the memory usage after conversion
- Understand why converting string columns to categorical types is useful in ML

Key Pandas Concepts

.astype('category'), .memory_usage(), Data type optimization

```python
# Task 8 – Handling Categorical Data

# 1. Check memory usage before conversion
memory_before = df.memory_usage(deep=True).sum()
print("Memory usage before conversion:", memory_before, "bytes")

# 2. Convert 'split' column to categorical
df['split'] = df['split'].astype('category')

# 3. Check memory usage after conversion
memory_after = df.memory_usage(deep=True).sum()
```

```
print("Memory usage after conversion:", memory_after, "bytes")

# 4. Display the DataFrame info to verify
df.info()
```

```
Memory usage before conversion: 15414 bytes
Memory usage after conversion: 13473 bytes
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 42 entries, 0 to 41
Data columns (total 8 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   sentence         42 non-null     object
 1   start            42 non-null     int64
 2   end              42 non-null     int64
 3   split            42 non-null     category
 4   start_sec        42 non-null     float64
 5   end_sec          42 non-null     float64
 6   duration_sec     42 non-null     float64
 7   sentence_length  42 non-null     int64
dtypes: category(1), float64(3), int64(3), object(1)
memory usage: 2.6+ KB
```

Task 9: Combining Data (Merging and Concatenation)

Task Description

In this task, you will combine sentence-level data with video-level metadata.

Steps:

- Create a separate DataFrame df_info containing video-level metadata: url, date, presenter, and signer
- Add relevant metadata columns (date and presenter) to the sentence DataFrame using broadcasting

Key Pandas Concepts

DataFrame creation from scalar values, Broadcasting, pd.merge() (conceptually)

```
# Task 9 – Combining Data

# 1. Create a separate DataFrame for video-level metadata
df_info = pd.DataFrame({
    "url": [video_data["url"]],
```

```
    "date": [video_data["date"]],
    "presenter": [video_data["presenter"]],
    "signer": [video_data["signer"]]
})

# 2. Add date and presenter to the main sentence DataFrame using
broadcasting
df['date'] = video_data["date"]
df['presenter'] = video_data["presenter"]
df['signer'] = video_data["signer"]
#todo

# 3. Preview the updated DataFrame
df.head()

                                            sentence  start   end
split  \
0                              আসসালামু আলাইকুম    260   281
train
1  বিটিভি ও বিটিভি ওয়ার্ল্ডের দেশ ও জনপদের খবরের        ...    282   518
train
2  আজ পাবনার ঈশ্বরদী উপজেলা মুক্তিযোদ্ধা কমান্ডের        ...    592   742
train
3  তিনি গতকাল মাদারীপুর স্বাধীনতা অঙ্গনে মাদারীপু        ...   1361  1643
train
4  এই সময় বইমেলা উদযাপন কমিটির আহ্বায়ক সহ সরকার      ...   1643  1935
test

   start_sec    end_sec  duration_sec  sentence_length        date  \
0   8.666667   9.366667      0.700000               16  8 DEC 2017
1   9.400000  17.266667      7.866667              133  8 DEC 2017
2  19.733333  24.733333      5.000000              101  8 DEC 2017
3  45.366667  54.766667      9.400000              137  8 DEC 2017
4  54.766667  64.500000      9.733333              117  8 DEC 2017

       presenter        signer
0  ফেরদৌসী শম্পা  Shahnaz Rita
1  ফেরদৌসী শম্পা  Shahnaz Rita
2  ফেরদৌসী শম্পা  Shahnaz Rita
3  ফেরদৌসী শম্পা  Shahnaz Rita
4  ফেরদৌসী শম্পা  Shahnaz Rita
```

Task 10: Data Visualization Preparation (Value Counts & Binning)

Task Description

In this task, you will prepare the dataset for analysis and visualization by performing the following steps:

- Determine the top 3 most frequent words in the sentence column
- Create a new column duration_bin by grouping the duration_sec into three bins: Short (0-5s), Medium (5-10s), and Long (>10s)
- Use .value_counts() on the new duration_bin column to show the distribution of sentence lengths

Key Pandas Concepts

.str.lower(), .str.split(), value counting, pd.cut() for binning

```
# Task 10 – Data Visualization Preparation

from collections import Counter

# 1. Determine top 3 most frequent words
all_words = df['sentence'].str.lower().str.split().sum()  # Flatten
list of all words
word_counts = Counter(all_words)
top_3_words = word_counts.most_common(3)
print("Top 3 most frequent words:", top_3_words)

# 2. Create duration bins
bins = [0, 5, 10, df['duration_sec'].max()]
labels = ['Short', 'Medium', 'Long']
df['duration_bin'] = pd.cut(df['duration_sec'], bins=bins,
labels=labels, right=False)

# 3. Show distribution of sentence lengths by duration_bin
duration_distribution = df['duration_bin'].value_counts()
print("\nDistribution of sentence lengths:")
print(duration_distribution)
```

```
Top 3 most frequent words: [('□', 17), ('□□□', 9), ('□□', 8)]

Distribution of sentence lengths:
duration_bin
Medium    25
Short     10
```

```
Long       6
Name: count, dtype: int64
```

Summary

Tasks Completed:

1. Data Loading and Initial Inspection
2. Feature Extraction and Manipulation (Time Features)
3. Data Filtering (Conditional Selection)
4. Text Data Preprocessing (String Operations)
5. Aggregation and Grouping (Split Analysis)
6. Missing Value Analysis and Handling
7. Indexing and Hierarchical Data Access
8. Handling Categorical Data
9. Combining Data (Merging and Concatenation)
10. Data Visualization Preparation (Value Counts & Binning)

All tasks have been successfully completed using core Pandas operations essential for Machine Learning preprocessing.